

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №6
на тему
«УПРАВЛЕНИЕ ПОТОКАМИ, СРЕДСТВА СИНХРОНИЗАЦИИ»
БГУИР 6-05-0612-02 22

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Выполнение работы	4
2.1	Архитектура программы	4
2.2	Реализация слияния	4
2.3	Сборка и запуск	4
	Вывод	6
	Список использованных источников	7
	Приложение А (обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение подсистемы потоков *pthread*, основных особенностей функционирования и управления ими, а также средств взаимодействия потоков в операционной системе *Linux*.

В рамках лабораторной работы необходимо разработать многопоточную программу для обработки большого массива данных (сортировки). Программа должна реализовывать:

- разбиение массива на несколько фрагментов;
- сортировку каждого фрагмента в отдельном потоке;
- окончательную сборку (слияние) отсортированных частей в главном потоке.

Параметры программы (размер массива и количество потоков) задаются пользователем. Результатом работы должны быть сведения о времени выполнения и протокол корректности.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура программы

Программа реализована на языке *C* с использованием библиотеки *pthread*. Алгоритм работы основан на принципе «разделяй и властвуй», адаптированном для многопоточного выполнения [1].

2.1.1 Главный поток

Выполняет инициализацию и финальную сборку:

- считывает параметры командной строки;
- выделяет память под массив и заполняет его случайными числами;
- рассчитывает границы фрагментов для каждого рабочего потока;
- создает рабочие потоки с помощью функции *pthread_create* [2];
- ожидает завершения всех потоков через *pthread_join*
- выполняет слияние отсортированных фрагментов;
- выводит статистику времени работы.

2.1.2 Рабочие потоки

Каждый поток обрабатывает свой диапазон индексов массива:

- получает структуру *sort_args_t* с указателем на массив и границами (начало и конец);
- вызывает стандартную функцию *qsort* для быстрой сортировки выделенного фрагмента [3];
- завершает выполнение, возвращая управление главному потоку.

2.2 Реализация слияния

После завершения параллельной сортировки фрагментов необходимо объединить их. В данной работе реализован алгоритм *N*-путевого слияния, который за один проход по всем элементам формирует итоговый отсортированный массив. Это обеспечивает линейную сложность $O(n)$ на этапе сборки, что эффективно для больших объемов данных [4].

2.3 Сборка и запуск

Проект содержит *Makefile* для упрощения компиляции. Сборка выполняется командой *make*. Программа *sort_test* принимает размер массива и количество потоков.

Таблица 2.1 – Пример выполнения тестов с разным количеством потоков

Потоков	Размер массива	Время (сек)
1	1,000,000	0.184
2	1,000,000	0.096

4	1,000,000	0.063
8	1,000,000	0.059

Как видно из таблицы 2.1, использование многопоточности позволяет существенно сократить время обработки данных. Наибольший прирост наблюдается при переходе от одного к двум и четырем потокам.

ВЫВОД

В ходе лабораторной работы были изучены основы работы с потоками в *POSIX*-совместимых системах. Были освоены функции создания потоков (`pthread_create`) и синхронизации через ожидание завершения (`pthread_join`) [2]. Реализованный алгоритм многопоточной сортировки продемонстрировал практическую эффективность параллельных вычислений: время выполнения задачи сократилось почти в 3 раза при использовании 4-8 потоков на много-ядерной системе. Были закреплены навыки работы с разделяемой памятью и передачи аргументов в потоковые функции.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Jayaweera, M. Parallel Merge Sort with Pthreads [Электронный ресурс] / M. Jayaweera. – Режим доступа: <https://malithjayaweera.com/2019/02/parallel-merge-sort/>. – Дата доступа: 07.05.2026.

[2] pthreads(7) — Linux manual page [Электронный ресурс]. – Режим доступа: <https://man7.org/linux/man-pages/man7/pthreads.7.html>. – Дата доступа: 07.05.2026.

[3] The GNU C Library: POSIX Threads [Электронный ресурс]. – Режим доступа: https://www.gnu.org/software/libc/manual/html_node/POSIX-Threads.html. – Дата доступа: 07.05.2026.

[4] Multi-threaded Merge Sort — CS 241 [Электронный ресурс]. – Режим доступа: <https://courses.grainger.illinois.edu/cs241/fa2012/assignments/mergesort/main.html>. – Дата доступа: 07.05.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код файла main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <string.h>

typedef struct {
    int *array;
    int start;
    int end;
} sort_args_t;

int compare_ints(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

void *sort_fragment(void *arg) {
    sort_args_t *args = (sort_args_t *)arg;
    qsort(args->array + args->start, args->end - args->start, sizeof(int),
    compare_ints);
    return NULL;
}

void merge_sorted_fragments(int *array, int size, int threads_count) {
    int *temp = malloc(size * sizeof(int));
    int *indices = calloc(threads_count, sizeof(int));
    int *chunk_ends = malloc(threads_count * sizeof(int));

    int chunk_size = size / threads_count;
    for (int i = 0; i < threads_count; i++) {
        indices[i] = i * chunk_size;
        chunk_ends[i] = (i == threads_count - 1) ? size : (i + 1)
* chunk_size;
    }

    for (int i = 0; i < size; i++) {
        int min_val = 0;
        int min_idx = -1;

        for (int j = 0; j < threads_count; j++) {
            if (indices[j] < chunk_ends[j]) {
                if (min_idx == -1 || array[indices[j]] < min_val) {
                    min_val = array[indices[j]];
                    min_idx = j;
                }
            }
        }
        temp[i] = min_val;
        indices[min_idx]++;
    }
}
```

```

    }

    memcpy(array, temp, size * sizeof(int));
    free(temp);
    free(indices);
    free(chunk_ends);
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <array_size> <threads_count>\n", argv[0]);
        return 1;
    }

    int size = atoi(argv[1]);
    int threads_count = atoi(argv[2]);

    if (size <= 0 || threads_count <= 0) {
        printf("Invalid arguments\n");
        return 1;
    }

    int *array = malloc(size * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < size; i++) {
        array[i] = rand() % 1000000;
    }

    pthread_t *threads = malloc(threads_count * sizeof(pthread_t));
    sort_args_t *args = malloc(threads_count * sizeof(sort_args_t));

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    int chunk_size = size / threads_count;
    for (int i = 0; i < threads_count; i++) {
        args[i].array = array;
        args[i].start = i * chunk_size;
        args[i].end = (i == threads_count - 1) ? size : (i + 1)
* chunk_size;

        if (pthread_create(&threads[i], NULL, sort_fragment, &args[i]) != 0) {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < threads_count; i++) {
        pthread_join(threads[i], NULL);
    }

    merge_sorted_fragments(array, size, threads_count);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec -
start.tv_nsec) / 1000000000.0;

```

```

printf("Array size: %d\n", size);
printf("Threads: %d\n", threads_count);
printf("Time elapsed: %.6f seconds\n", elapsed);

int sorted = 1;
for (int i = 0; i < size - 1; i++) {
    if (array[i] > array[i + 1]) {
        sorted = 0;
        break;
    }
}
printf("Result: %s\n", sorted ? "SUCCESS (Sorted)" : "FAILURE (Not
sorted)");

free(array);
free(threads);
free(args);

return 0;
}

```

Листинг А.2 – Код файла Makefile

```

CC = gcc
CFLAGS = -Wall -Wextra -pthread
TARGET = sort_test

all: $(TARGET)

$(TARGET): main.c
    $(CC) $(CFLAGS) main.c -o $(TARGET)

clean:
    rm -f $(TARGET)

test: all
    @echo "Running test with 1 thread (baseline)..."
    ./$(TARGET) 1000000 1
    @echo "\nRunning test with 2 threads..."
    ./$(TARGET) 1000000 2
    @echo "\nRunning test with 4 threads..."
    ./$(TARGET) 1000000 4
    @echo "\nRunning test with 8 threads..."
    ./$(TARGET) 1000000 8

```